

RAMA: INFORMÁTICA	CICLO: DESENVOLVIMIENTO DE APLICACIONES MULTIPLATAFORMA
MÓDULO: ACCESO A DATOS	CURSO: 2º
PROFESOR: IAGO DOVAL IGLESIAS	
AVALIACIÓN: RECUPERACIÓN	GRUPO: ÚNICO DATA: Junio 2026
NOMBRE ALUMNO:	DNI:

El examen consta de **dos partes** valoradas en **5 puntos cada una**. Para aprobar hay que obtener un mínimo de **5 puntos** en total con al menos **2 puntos** en cada parte. Sube un único fichero comprimido con todos los archivos utilizados: **1erApellido_2oApellido_Nombre**.

Trata las excepciones adecuadamente y libera todos los recursos (-0.10 pt por elemento no tratado). Muestra un ejemplo de invocación en cada ejercicio (-0.15 pt si no se indica).

PARTE 1 – Ejercicios de Conectores (5 pt)

Usa el fichero **Conectores.java** adjunto y la base de datos proporcionada. Se adjunta también el conector necesario.

- 1) **(1.20 pt)** Crea un método, usando **sentencias preparadas**, que permita mostrar el id, nombre, plataforma y precio de alquiler de los juegos cuyo stock de alquiler esté entre dos valores pasados como parámetros. El método debe devolver el número de resultados encontrados o -1 si se produce un error, y mostrar ese valor fuera del método.
- 2) **(1.10 pt)** Crea un método, usando **sentencias sin preparar**, que dado el nombre de un juego y un valor entero, cambie su stock de alquiler a ese valor. El método debe devolver: **1** si la actualización se realizó, **0** si no se realizó, **-1** si se produjo un error.
- 3) **(1.20 pt)** Crea un método, usando **sentencias preparadas**, que muestre ordenado por posición: la posición en la clasificación, el gap con el piloto precedente, el nombre de la escudería y el nombre del piloto, para los pilotos entre dos posiciones pasadas como parámetros. Si no puedes obtener el nombre de la escudería, muestra su código (-0.40 pt).
- 4) **(1.50 pt)** Crea un método al que se le pase el nombre de una tabla y muestre, para cada columna: el nombre (COLUMN_NAME), el tipo de datos (TYPE_NAME), si permite nulos (NULLABLE) y si es autoincrement (IS_AUTOINCREMENT). Para las dos últimas debe aparecer **"Sí"** o **"No"**. Usa **DatabaseMetaData**.

PARTE 2 – Ejercicios de Servicios Web (5 pt)

Condiciones generales:

- a) El path inicial del servicio web será **api**. (-0.10 pt si se especifica otro).
 - b) Si no se especifica un path en el ejercicio, se usará el del punto anterior.
 - c) Todos los métodos deben devolver un objeto tipo **Response**. En caso de error, devuelve un código de error y un mensaje explicativo.
 - d) Si no se especifica lo contrario, se produce y/o consume **JSON y XML**. Para listas, solo se produce JSON. (-0.10 pt si se especifica otro).
 - e) Crea un **ArrayList estático** llamado **escuderias** para los ejercicios 5 y 6. La base de datos solo se usa en los ejercicios 7 y 8.
- 5) **(1.10 pt)** Un método que cree y añada al ArrayList **escuderias** una nueva escudería. Los parámetros *sede* y *año de debut* se pasan como **QueryParams**; el resto se cubren a mano (nada en el Body). El valor por defecto de sede será **"Desconocida"**. Devuelve la escudería creada. No hay que controlar códigos repetidos.
 - 6) **(1.20 pt)** Un método en el path **"{motor}"** que borre del ArrayList **escuderias** todas las escuderías cuyo motorista contenga el valor pasado. Debe devolver la lista de escuderías resultante.

- 7) **(1.10 pt)** Un método que devuelva todos los juegos presentes en la base de datos en los que el *precioalquiler* sea nulo.
- 8) **(1.60 pt)** Un método en el path "**{actualizar}**" que, al pasarle una escudería mediante un JSON, la actualice si ya existe su *codEscuderia* en la base de datos. Si no existe, que la inserte. El id se pasará como **QueryParam**. Si no puedes insertar cuando no existe, puede devolver un error (-0.80 pt).